

ENSF 380

TOPIC 21: DATA STRUCTURES

Data Structures

Collection: group of objects

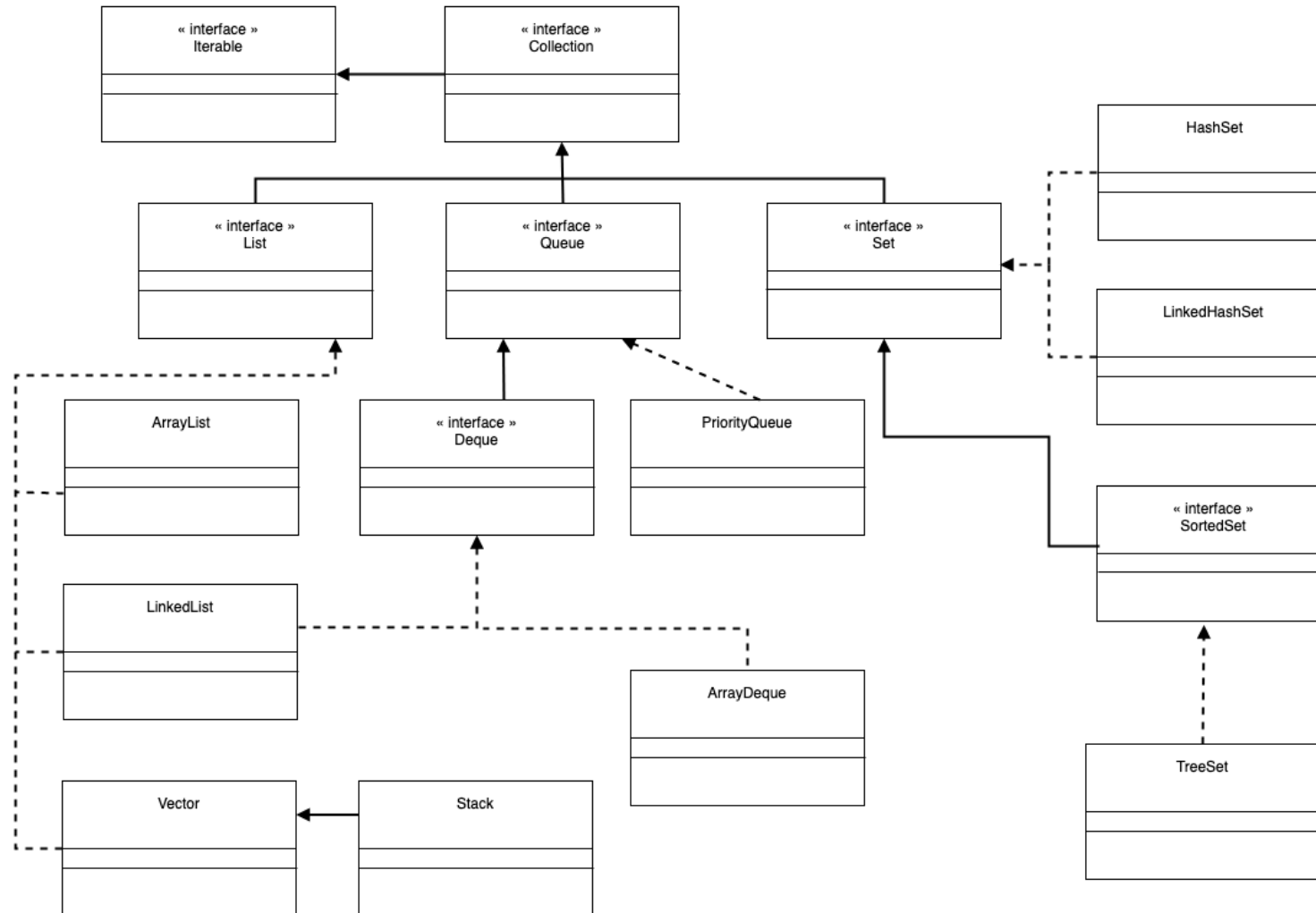
- Bag: allows duplicates, unordered
- List: allows duplicates, ordered
 - Queue: FIFO
 - Stack: LIFO
- Set: unique elements

Java Collections Framework

Java Collections Framework (JCF)

- Contains interfaces and classes which allow data to be represented as sets, lists, bags, etc.
- Contains useful algorithms for manipulation of data
 - `add()`
 - `equals()`
 - `contains()`
 - `isEmpty()`
 - `removeAll()`
 - `size()`
 - etc.
- Part of the `java.util` package
 - `import java.util.*`

Java Collections Framework



Types of Collections

ArrayList

- Random access
- Adding and removing from the end
- Allows null elements



ArrayDeque

- Random access
- Adding and removing from the start or end
- Does not allow null elements



LinkedList

- Adding and removing from the middle
- Allows null elements

Vector

- Thread-safe
- Similar to ArrayList

Types of Collections

PriorityQueue

- Prioritized ordering
- Does not allow null elements

TreeSet

- Unique elements
- Does not allow null elements

Stack

- LIFO
- Thread-safe
- Extends Vector
- Allows null elements, but they are not recommended

Types of Collections

HashSet

- Unordered
- Unique elements
- Allows one null element
- Search operations

LinkedHashSet

- Insertion order
- Adding and removing from the start or end
- Allows one null element
- Search operations

Java Collections Framework Reference

Class	Order maintained	Allows duplicates	Allows null	Thread-safe
ArrayDeque	Insertion order	Yes	No	No
ArrayList	Insertion order	Yes	Yes	No
HashSet	Unordered	No	One	No
LinkedHashSet	Insertion order	No	One	No
LinkedList	Insertion order	Yes	Yes	No
PriorityQueue	Priority / Unordered	Yes	No	No
Stack	Insertion order	Yes	Yes (but avoid)	Yes
TreeSet	Ascending order	No	No	No
Vector	Insertion order	Yes	Yes	Yes

Collections: When to Use

Class	When to Use
ArrayDeque	Access random elements, add and remove at start or end of list, insertion order matters
ArrayList	Access random elements, add and remove at end of list, insertion order matters
HashSet	Searching
LinkedHashSet	Searching, insertion order matters
LinkedList	Access through iteration, add and remove in the middle of list, insertion order matters
PriorityQueue	Prioritized order
Stack	LIFO, thread-safe
TreeSet	Sorted, set (unique elements)
Vector	Access random elements, add and remove at end of list, insertion order matters, thread-safe

```
import java.util.*;

...

ArrayList<String> myCake = new ArrayList <String>();
myCake.add("Chocolate");
myCake.add("Vanilla");
myCake.add("Lemon");

Iterator<String> iter = myCake.iterator();
System.out.println("Cakes are: ");
while(iter.hasNext()) {
    System.out.println(iter.next());
}
```

```
String elem = myCake.get(1);  
System.out.println("Second cake element is: " + elem);  
System.out.println("I have this many cakes: " + myCake.size());  
  
myCake.clear();  
if(myCake.isEmpty()) {  
    System.out.println("All cakes were eaten.");  
}
```

ArrayList

01_ArrayList

```
myCake.add("Sponge");  
myCake.add("Bundt");  
myCake.add("Carrot");  
Collections.sort(myCake);  
  
System.out.println("I got more cakes: ");  
Object[] cakes = myCake.toArray();  
for(int i=0; i < cakes.length; i++) {  
    System.out.println(cakes[i]);  
}
```

```
I got more cakes:  
Bundt  
Carrot  
Sponge
```

HashSet

02_HashSet

```
Integer arrayYears[] = {2021, 1927, 1066};  
Set<Integer> years = new HashSet<>(Arrays.asList(arrayYears));  
years.add(1859);  
  
System.out.println("Years: ");  
Iterator <Integer> it = years.iterator();  
while (it.hasNext()) {  
    System.out.println(it.next());  
}
```

```
Years:  
1859  
2021  
1927  
1066
```

Collections

```
Interface<E> variableName = new CollectionType <> (...);
```

```
Iterator<T> iteratorName = object.iterator();
```

```
ArrayList<String> myCake = new ArrayList <>();
```

```
Set<Integer> years = new HashSet<>(Arrays.asList(arrayYears));
```

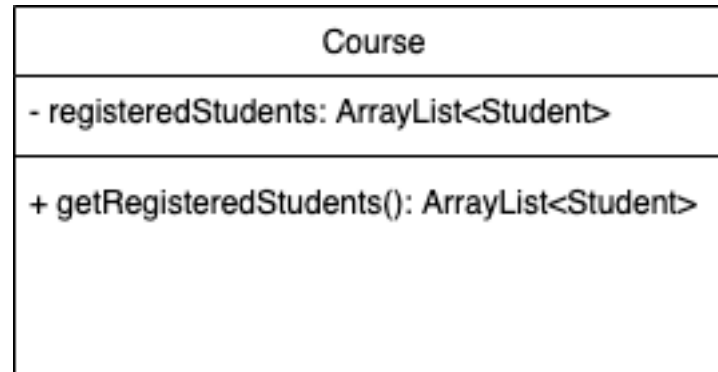
Exercise 21.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Exercise 21.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Collections in UML



Enums

Short for "enumerations"

Represents a set of constants

Can have attributes and methods

Are public, static, and final

Cannot extend

Can implement interfaces

Enums

Enum Declaration

```
public enum Suits {  
    SPADES,  
    HEARTS,  
    CLUBS,  
    DIAMONDS  
}
```

JVM

```
final class Suits extends Enum <Suits> {  
    public final static Suits SPADES =  
        new Suits();  
    public final static Suits HEARTS =  
        new Suits();  
    public final static Suits CLUBS =  
        new Suits();  
    public final static Suits DIAMONDS =  
        new Suits();  
}
```

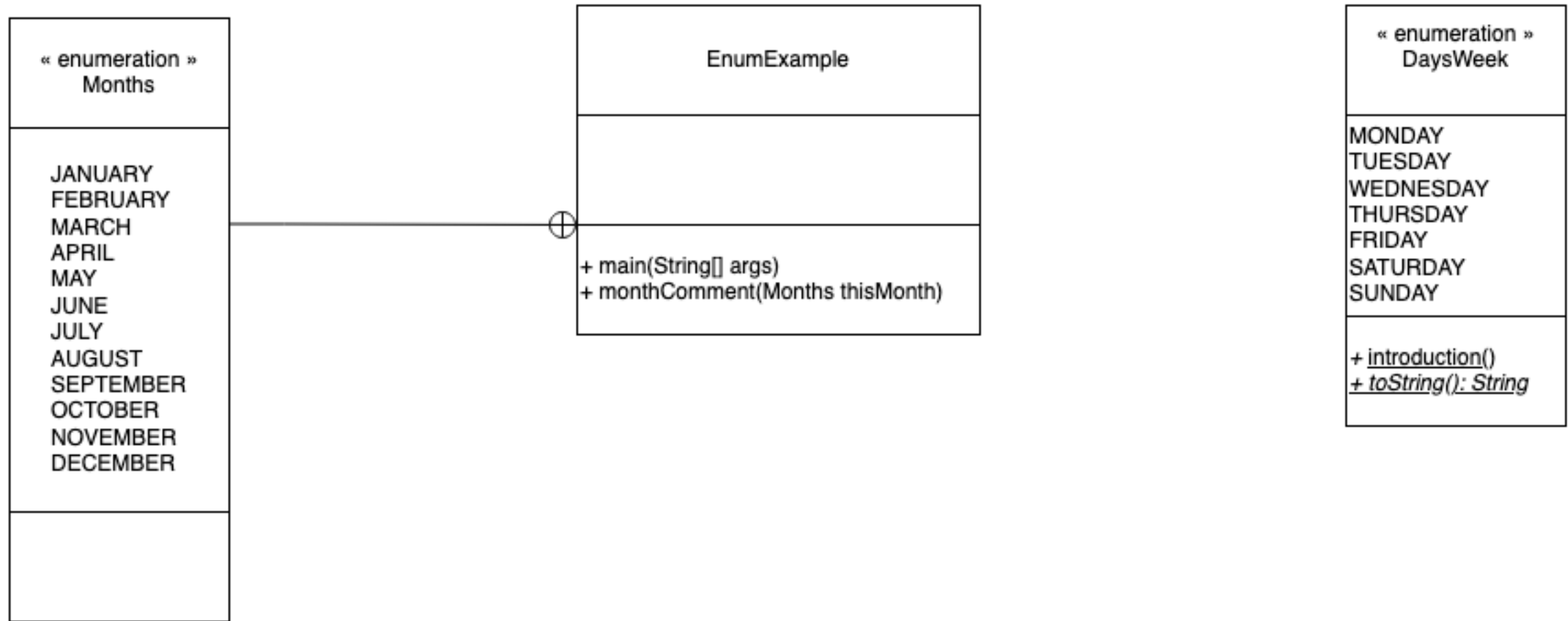
```
public class EnumExample {  
    enum Months {  
        JANUARY,  
        FEBRUARY,  
  
        NOVEMBER,  
        DECEMBER  
    }  
  
    public static void main(String[] args) {  
        for (Months month: Months.values()) {  
            printCommentOnMonth(month);  
        }  
    }  
}
```

```
public static void printCommentOnMonth(Months thisMonth) {  
  
    switch(thisMonth) {  
        case JANUARY:  
            System.out.println(thisMonth + ": Start of the term.");  
            break;  
        case FEBRUARY:  
            System.out.println(thisMonth + ": It is cold in Calgary.");  
            break;  
        case MARCH:  
            System.out.println(thisMonth + ": Winter semester is almost done.");  
            break;  
    }  
}
```

```
enum DaysWeek {  
    MONDAY {  
        public String toString() {  
            return "Monday";  
        }  
    },  
    ...  
};  
  
public abstract String toString();  
  
public static void introduction() {  
    System.out.println("Method introduction called from DaysWeek");  
}  
}
```

```
DaysWeek.introduction();  
  
DaysWeek day = DaysWeek.SATURDAY;  
  
day.introduction();  
  
int i = day.ordinal();  
System.out.println(day.toString() + " is at position " + i);
```

Enums in UML



Exercise 21.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Know terms for describing groups of objects:

- Collection
- Bag
- List
- Queue
- Stack
- Set

Understand the characteristics of ArrayList, ArrayDeque, LinkedList, Vector, PriorityQueue, TreeSet, Stack, HashSet, LinkedHashSet, HashMap

Understand the use case for various collections

Characteristics and implementation of Enum

Summary: Skills

Be able to work with various collections in the Java Collections Framework

Be able to use Enums

Additional Information

Related reading

- Volume 1, Chapter 9: Collections (primarily 9.1-9.3)

Reminder

- You are expected to adhere to the stylistic convention of using uppercase for enum elements